CURSO ITF DE TESTING MANUAL

Base de Datos de Recursos Humanos

Gestión de la estructura de una base de datos y consultas SQL

Resumen del proyecto

Elemento	Detalles
Estudiante	Valentina Hatos
Rol	QA Tester
Aplicación	Base de datos MySQL “humanresources”
Módulo probado	Estructura de la base de datos, cambios de datos y consultas SQL
Herramientas	MySQL Workbench
Fecha	Septiembre de 2022

Índice

1. Introducción....3

2. Creación de las bases de datos y las tablas....3

3. Modificación de la estructura de las tablas....4

4. Inserción de datos....5

5. Consultas....5

5.1 Selección y ordenación....5

5.2 Filtros y funciones....5

5.3 Modificación de los datos....6

6. Vinculación de las tablas....6

7. Consultas con joins....7

8. Conclusión....7

1. Introducción

Este proyecto es un ejercicio de SQL del curso ITF de Testing Manual. Creé una base de datos para un departamento de recursos humanos. La base de datos guarda los empleados y las recompensas que reciben.

En este ejercicio, me centré en la estructura de la base de datos. Creé y eliminé bases de datos y tablas, añadí y eliminé columnas, y modifiqué claves. Después inserté datos, los modifiqué y escribí consultas, incluidos joins entre dos tablas. El ejercicio tiene 22 tareas.

2. Creación de las bases de datos y las tablas

Tareas 1 y 2: Creé dos bases de datos, humanresources y marketing. Después eliminé la base de datos marketing con DROP DATABASE.

```
create database humanresources;
create database marketing;

drop database marketing;
```

Tarea 3: Creé tres tablas en la base de datos humanresources.

Tabla	Qué guarda
employee	El id, el nombre, el salario y la fecha de incorporación de cada empleado
reward	Las recompensas de los empleados: la fecha y el importe
holidays	El nombre y la fecha de cada día festivo

La columna Employee_id es la clave primaria y aumenta automáticamente.

```
create table employee (
Employee_id int auto_increment primary key,
First_name varchar(50),
Last_name varchar(50),
Salary int,
Joining_date date
);

create table reward (
Employee_ref_id int,
date_reward date,
amount int
);

create table holidays (
holiday_name varchar(50),
holiday_date date
);
```

3. Modificación de la estructura de las tablas

Tarea 4: Añadí dos columnas nuevas a la tabla employee: Birthdate y Departament.

```
alter table employee add Birthdate date;
alter table employee add Departament varchar(50);
```

Tarea 5: Eliminé la columna Birthdate.

```
alter table employee drop Birthdate;
```

Tarea 6: Eliminé la tabla holidays.

```
drop table holidays;
```

Después de estos cambios, la base de datos tiene dos tablas: employee y reward.

4. Inserción de datos

Tarea 7: Inserté 6 empleados de cuatro departamentos: IT, Banking, Insurance y Finance. Después actualicé el salario del primer empleado. También inserté 4 recompensas.

```
insert into employee (First_name, Last_name, Salary, Joining_date, Departament)
values ('Jerry', 'Kansxo', '6000000', '2019-01-15', 'IT'), ('Philip', 'Jose',
'8900000', '2019-02-05', 'Banking'), ...

update employee
set Salary = '1000000'
where Employee_id = '1';

insert into reward (date_reward, amount)
values ('2019-05-11', 1000), ('2019-02-15', 5000), ('2019-04-22', 2000), ('2019-
06-20', 8000);
```

Tarea 8: Revisé las dos tablas con SELECT *. Después convertí Employee_ref_id en la clave primaria de la tabla reward, con numeración automática.

```
alter table reward
modify Employee_ref_id int auto_increment primary key;
```

5. Consultas

5.1 Selección y ordenación

Tarea 9: Seleccioné el nombre y el apellido de todos los empleados.

```
select First_name, Last_name from employee;
```

Tarea 10: Ordené los empleados por nombre.

```
select * from employee order by First_name;
```

Tarea 11: Ordené los empleados por nombre (ascendente) y después por salario (descendente).

```
select * from employee order by First_name asc, salary desc;
```

Tarea 12: Seleccioné los empleados con el nombre “Bob”.

```
select * from employee
where First_name = 'Bob';
```

5.2 Filtros y funciones

Tarea 13: Seleccioné los empleados con un salario mayor de 3.000.000.

```
select * from employee
where salary > 3000000;
```

La consulta devuelve dos empleados: Philip Jose y Alex chreketo.

Tarea 14: Seleccioné los empleados que se incorporaron antes del 1 de marzo de 2019.

```
select * from employee
where Joining_date < '2019-03-01';
```

La consulta devuelve cuatro empleados.

Tarea 15: Conté los empleados.

```
select count(*) from employee;
```

El resultado es 6.

Tarea 16: Calculé el salario medio.

```
select avg(Salary) from employee;
```

El salario medio es de unos 3.221.667.

5.3 Modificación de los datos

Tarea 17: Actualicé el salario del empleado con el apellido Kansxo.

```
update employee
set Salary = 8880000
where Last_name = 'Kansxo';
```

Tarea 18: Eliminé todos los empleados del departamento Banking.

```
delete from employee
where Departament = 'Banking';
```

Se eliminaron dos empleados: Philip Jose y Yohan Soso.

Tarea 19: Seleccioné los dos empleados con los salarios más altos.

```
select * from employee order by Salary desc limit 2;
```

Después de los cambios de las tareas 17 y 18, la consulta devuelve a Jerry Kansxo y Alex chreketo.

6. Vinculación de las tablas

Después, vinculé la tabla reward a la tabla employee mediante una clave foránea. La columna Employee_ref_id de la tabla reward hace referencia a la columna Employee_id de la tabla employee.

Para ello, primero preparé los datos. Vacié las dos tablas con TRUNCATE y volví a insertar los datos. Esta vez, inserté 7 empleados, incluido uno nuevo, Bob Kinto.

```
truncate table employee;
insert into employee (First_name, Last_name, Salary, Joining_date, Departament)
values ('Bob', 'Kinto', 1000000, '2019-01-20', 'Finance'), ('Jerry', 'Kansxo',
'6000000', '2019-01-15', 'IT'), ...
```

Un empleado puede recibir más de una recompensa. Por eso, Employee_ref_id no puede ser la clave primaria de la tabla reward. Eliminé la clave primaria y la numeración automática de esta columna. Después vinculé las recompensas a los empleados 1, 2, 3 y 1, y añadí la clave foránea.

```
alter table reward
drop primary key;

alter table reward
modify Employee_ref_id int;

alter table reward
add foreign key (Employee_ref_id) references employee (Employee_id);
```

Verifiqué el resultado con sentencias SELECT * y DESC.

7. Consultas con joins

Tarea 20: Para cada recompensa, calculé la diferencia entre la fecha de la recompensa y la fecha de incorporación del empleado.

```
select r.date_reward - e.Joining_date
from employee e inner join reward r on e.Employee_id = r.Employee_ref_id;
```

Uní las dos tablas con INNER JOIN, mediante la clave foránea. Utilicé los alias “e” y “r” para que la consulta sea más corta.

Tarea 21: Seleccioné el nombre de los empleados y el importe de sus recompensas, solo para las recompensas mayores de 2000.

```
select e.First_name, r.amount
from employee e inner join reward r on e.Employee_id = r.Employee_ref_id
having amount > 2000;
```

Utilicé HAVING para filtrar el resultado.

Tarea 22: Seleccioné el nombre de los empleados y el importe de todas sus recompensas.

```
select e.First_name, r.amount
from employee e inner join reward r on e.Employee_id = r.Employee_ref_id;
```

El script SQL completo se puede consultar en el repositorio del proyecto (humanresource database.sql). El modelo de la base de datos está disponible en humanresource_database_model.mwb.

8. Conclusión

En este ejercicio, practiqué el trabajo con la estructura de una base de datos. Creé y eliminé bases de datos y tablas, añadí y eliminé columnas, y modifiqué claves primarias y foráneas. También inserté, actualicé y eliminé datos.

La parte más importante fue vincular las dos tablas. Aprendí que una columna de clave foránea debe permitir valores repetidos cuando un registro (un empleado) puede estar vinculado a varios registros (recompensas). Después de vincular las tablas, escribí consultas con joins para combinar datos de las dos.